0206. Reverse Linked List

Easy | Category: Linked List | LeetCode: https://leetcode.com/problems/reverse-l

Problem

- Reverse a singly linked list and return the new head.
- Example: 1→2→3→4→5 becomes 5→4→3→2→1.

Intuition

- Walk the list once and flip each next pointer to point backwards.
- We need three references: prev (already reversed part), cur, and next (saved before we overwrite cur.next).

Brute Force

- Copy values into an array and rebuild: O(n) extra space.

Optimal Approach

- Prev = null, cur = head.
- Loop: next = cur.next; cur.next = prev; prev = cur; cur = next.
- Return prev.
- Recursive version: reverse(head.next), then head.next.next = head; head.next = null.

Worked Example

```
1->2->3.

prev=null cur=1 : 1->null          prev=1 cur=2
prev=1    cur=2 : 2->1             prev=2 cur=3
prev=2    cur=3 : 3->2             prev=3 cur=null
return 3 (3->2->1)
```

Java Solution

```java
public class ReverseLinkedList {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null, cur = head;
        while (cur != null) {
            ListNode next = cur.next;
            cur.next = prev;
            prev = cur;
            cur = next;
        }
        return prev;
    }

    public ListNode reverseRecursive(ListNode head) {
        if (head == null || head.next == null) return head;
        ListNode newHead = reverseRecursive(head.next);
        head.next.next = head;
        head.next = null;
        return newHead;
    }
}
```

Complexity

- Iterative: O(n) time, O(1) space. Recursive: O(n) stack.

Pitfalls

- Save next before overwriting cur.next, or you lose the rest of the list.